

LAPORAN PRATIKUM STRUKTUR DATA

“ALGORITMA SORTING”



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : Dr. WAHYUDI,

S.T, M.T

ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI
INFORMASI DEPARTEMEN

INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data Algoritma Sorting serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori.....	2
2.2 Penjelasan Kode Program.....	3
2.2.1 Class InsertionSort_2511531007	3
2.2.2 Class SelectionSort_2511531007	4
2.2.3 Class BubbleSort_2511531007	6
2.2.4 Class InsertionSortGUI_2511531007	7
2.3 Tugas Praktikum	11
2.3.1 Class Mahasiswa_2511531007.....	11
2.3.2 Class SortingMahasiswaGUI_2511531007	12
BAB III PENUTUP	18
3.1 Kesimpulan	18
3.2 Saran	18
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data dan algoritma merupakan bagian penting dalam pemrograman yang digunakan untuk mengelola serta mengolah data secara efisien. Salah satu algoritma yang sering digunakan adalah algoritma sorting, yaitu algoritma yang berfungsi untuk mengurutkan data berdasarkan urutan tertentu. Proses pengurutan data dapat mempermudah pencarian, pengelompokan, dan pengolahan data dalam berbagai aplikasi.

Terdapat beberapa metode sorting yang umum digunakan, seperti Insertion Sort, Selection Sort, dan Bubble Sort. Masing-masing algoritma memiliki cara kerja yang berbeda dalam mengurutkan data. Pada praktikum ini dilakukan implementasi ketiga algoritma sorting tersebut menggunakan bahasa pemrograman Java untuk mengurutkan nama mahasiswa secara alfabetis.

Program yang dibuat menggunakan Java GUI (Swing) sehingga pengguna dapat memasukkan data mahasiswa, memilih algoritma sorting yang akan digunakan, serta melihat proses pengurutan secara bertahap. Melalui praktikum ini diharapkan dapat memahami konsep, cara kerja, serta perbedaan implementasi algoritma Insertion Sort, Selection Sort, dan Bubble Sort dalam pengolahan data bertipe String.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dan cara kerja algoritma Insertion Sort, Selection Sort, dan Bubble Sort dalam proses pengurutan data.
2. Mengimplementasikan ketiga algoritma sorting menggunakan bahasa pemrograman Java serta menampilkan proses pengurutannya melalui aplikasi berbasis GUI (Swing).

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Meningkatkan pemahaman mengenai perbedaan proses dan karakteristik algoritma Insertion Sort, Selection Sort, dan Bubble Sort dalam pengurutan data.
2. Menambah keterampilan dalam mengimplementasikan algoritma sorting dan mengembangkan aplikasi sederhana berbasis Java GUI (Swing).

BAB II PEMBAHASAN

2.1 Dasar Teori

Algoritma sorting merupakan salah satu metode yang digunakan untuk mengurutkan data berdasarkan urutan tertentu, baik secara ascending (menaik) maupun descending (menurun). Proses pengurutan data sangat penting dalam pengolahan informasi karena dapat mempermudah pencarian, pengelompokan, dan pengelolaan data. Dengan data yang telah terurut, proses pengolahan data menjadi lebih efisien dan mudah dipahami.

Terdapat berbagai jenis algoritma sorting, di antaranya Insertion Sort, Selection Sort, dan Bubble Sort. Insertion Sort bekerja dengan cara menyisipkan setiap elemen ke posisi yang sesuai pada bagian data yang telah terurut. Selection Sort bekerja dengan mencari elemen terkecil dari bagian data yang belum terurut, kemudian menukarkannya ke posisi yang tepat. Sementara itu, Bubble Sort bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila urutannya tidak sesuai. Ketiga algoritma tersebut memiliki mekanisme yang berbeda, tetapi sama-sama bertujuan untuk menghasilkan data yang tersusun secara teratur.

Dalam implementasinya, algoritma sorting dapat digunakan untuk mengurutkan berbagai jenis data, termasuk data bertipe String. Pada bahasa pemrograman Java, proses perbandingan data String dapat dilakukan menggunakan method `compareToIgnoreCase()`, sehingga perbandingan tidak dipengaruhi oleh penggunaan huruf kapital maupun huruf kecil. Penggunaan algoritma sorting dalam praktikum membantu memahami konsep pengurutan data serta penerapan logika algoritma dalam pemrograman.

2.2 Penjelasan Kode Program

2.2.1 Class InsertionSort_2511531007

```
1 package pekan7_2511531007;
2
3 public class InsertionSort_2511531007 {
4     public static void insertionSort_1007(int[] arr_1007) {
5         int n_1007 = arr_1007.length;
6         for (int i_1007 = 1; i_1007 < n_1007; i_1007++) {
7             int key_1007 = arr_1007[i_1007];
8             int j_1007 = i_1007 - 1;
9             while (j_1007 >= 0 && arr_1007[j_1007] > key_1007) {
10                 arr_1007[j_1007 + 1] = arr_1007[j_1007];
11                 j_1007--;
12             }
13             arr_1007[j_1007 + 1] = key_1007;
14         }
15     }
16
17     public static void main(String[] args) {
18         int arr_1007[] = { 23, 78, 45, 8, 32, 56, 1 };
19         int n_1007 = arr_1007.length;
20         System.out.printf("array yang belum terurut:\n");
21         for (int i_1007 = 0; i_1007 < n_1007; i_1007++)
22             System.out.print(arr_1007[i_1007] + " ");
23         System.out.println("");
24         insertionSort_1007(arr_1007);
25         System.out.printf("array yang terurut:\n");
26         for (int i_1007 = 0; i_1007 < n_1007; i_1007++)
27             System.out.print(arr_1007[i_1007] + " ");
28         System.out.println("");
29     }
30 }
31 }
```

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma Insertion Sort. Algoritma ini bekerja dengan cara mengambil satu per satu elemen dari array, kemudian menempatkannya pada posisi yang tepat di bagian array yang sudah terurut. Proses ini dilakukan secara berulang hingga seluruh elemen dalam array tersusun secara berurutan dari nilai terkecil ke terbesar.

Di dalam class InsertionSort_2511531007 terdapat method insertionSort_1007(int[] arr_1007) yang berfungsi untuk melakukan proses pengurutan data. Variabel n_1007 digunakan untuk menyimpan jumlah elemen yang terdapat pada array. Selanjutnya terdapat perulangan for yang dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut.

Pada setiap perulangan, nilai elemen yang sedang diproses disimpan ke dalam variabel key_1007. Variabel ini berfungsi sebagai data yang akan disisipkan ke posisi yang sesuai. Kemudian variabel j_1007 digunakan untuk menunjuk indeks elemen sebelumnya yang akan dibandingkan dengan key_1007.

Proses perbandingan dilakukan menggunakan perulangan while. Selama nilai pada `arr_1007[j_1007]` lebih besar daripada `key_1007`, elemen tersebut akan digeser satu posisi ke kanan. Setelah ditemukan posisi yang sesuai atau indeks sudah mencapai batas awal array, nilai `key_1007` ditempatkan pada posisi `j_1007 + 1`. Dengan cara ini, bagian array yang sudah diproses akan selalu berada dalam kondisi terurut.

Pada method `main()`, dibuat sebuah array `arr_1007` yang berisi data {23, 78, 45, 8, 32, 56, 1}. Program kemudian menampilkan isi array sebelum diurutkan menggunakan perulangan `for`. Setelah itu method `insertionSort_1007()` dipanggil untuk mengurutkan data pada array. Setelah proses pengurutan selesai, program kembali menampilkan isi array sehingga pengguna dapat melihat hasil perubahan dari data yang belum terurut menjadi data yang sudah terurut secara ascending (dari kecil ke besar).

2.2.2 Class SelectionSort_2511531007

```
1 package pekan7_2511531007;
2
3 public class SelectionSort_2511531007 {
4     public static void selectionSort_1007(int[] arr_1007) {
5         int n_1007 = arr_1007.length;
6         for (int i_1007 = 0; i_1007 < n_1007; i_1007++) {
7             int minIndex_1007 = i_1007;
8             for (int j_1007 = i_1007 + 1; j_1007 < n_1007; j_1007++) {
9                 if (arr_1007[j_1007] < arr_1007[minIndex_1007]) {
10                     minIndex_1007 = j_1007;
11                 }
12             }
13             int temp_1007 = arr_1007[i_1007];
14             arr_1007[i_1007] = arr_1007[minIndex_1007];
15             arr_1007[minIndex_1007] = temp_1007;
16         }
17     }
18
19     public static void main(String[] args) {
20         int arr_1007[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n_1007 = arr_1007.length;
22         System.out.printf("array yang belum terurut:\n");
23         for (int i_1007 = 0; i_1007 < n_1007; i_1007++)
24             System.out.print(arr_1007[i_1007] + " ");
25         System.out.println("");
26         selectionSort_1007(arr_1007);
27         System.out.printf("array yang terurut:\n");
28         for (int i_1007 = 0; i_1007 < n_1007; i_1007++)
29             System.out.print(arr_1007[i_1007] + " ");
30         System.out.println("");
31     }
32 }
33 }
```

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma Selection Sort. Algoritma ini bekerja dengan cara mencari elemen dengan nilai paling kecil pada bagian array yang belum terurut, kemudian menukarkannya dengan elemen

yang berada pada posisi saat ini. Proses tersebut dilakukan secara berulang hingga seluruh elemen array tersusun secara berurutan dari nilai terkecil ke terbesar.

Di dalam class `SelectionSort_2511531007` terdapat method `selectionSort_1007(int[] arr_1007)` yang berfungsi untuk melakukan proses pengurutan data. Variabel `n_1007` digunakan untuk menyimpan jumlah elemen yang terdapat pada array. Selanjutnya terdapat perulangan `for` yang digunakan untuk menentukan posisi elemen yang akan diisi dengan nilai terkecil pada setiap tahap pengurutan.

Pada setiap perulangan, variabel `minIndex_1007` diinisialisasi dengan nilai `i_1007`. Variabel ini berfungsi untuk menyimpan indeks elemen dengan nilai terkecil yang ditemukan. Kemudian perulangan `for` kedua digunakan untuk membandingkan elemen-elemen setelah posisi `i_1007`. Jika ditemukan elemen yang nilainya lebih kecil dari elemen pada `minIndex_1007`, maka nilai `minIndex_1007` akan diperbarui sesuai indeks elemen tersebut.

Setelah seluruh elemen pada bagian yang belum terurut selesai diperiksa, dilakukan proses pertukaran data menggunakan variabel `temp_1007` sebagai variabel sementara. Nilai pada posisi `i_1007` disimpan terlebih dahulu ke dalam `temp_1007`, kemudian nilai pada `minIndex_1007` dipindahkan ke posisi `i_1007`. Setelah itu nilai yang tersimpan pada `temp_1007` ditempatkan kembali ke posisi `minIndex_1007`. Dengan cara ini, elemen terkecil akan berpindah ke posisi yang seharusnya pada setiap tahap pengurutan.

Pada method `main()`, dibuat sebuah array `arr_1007` yang berisi data {23, 78, 45, 8, 32, 56, 1}. Program terlebih dahulu menampilkan isi array sebelum dilakukan pengurutan menggunakan perulangan `for`. Selanjutnya method `selectionSort_1007()` dipanggil untuk mengurutkan data pada array. Setelah proses pengurutan selesai, program kembali menampilkan isi array sehingga pengguna dapat melihat hasil perubahan dari data yang belum terurut menjadi data yang sudah terurut secara ascending (dari kecil ke besar).

2.2.3 Class BubbleSort_2511531007

```
1 package pekan7_2511531007;
2
3 public class BubbleSort_2511531007 {
4     public static void bubbleSort_1007(int[] arr_1007) {
5         int n_1007 = arr_1007.length;
6         for (int i_1007 = 0; i_1007 < n_1007; i_1007++) {
7             for (int j_1007 = 0; j_1007 < n_1007 - i_1007 - 1; j_1007++) {
8                 if (arr_1007[j_1007] > arr_1007[j_1007 + 1]) {
9                     int temp_1007 = arr_1007[j_1007];
10                    arr_1007[j_1007] = arr_1007[j_1007 + 1];
11                    arr_1007[j_1007 + 1] = temp_1007;
12                    // System.out.println("data: "+arr[j]+" "+arr[j+1]);
13                }
14            }
15        }
16    }
17
18    public static void main(String[] args) {
19        int arr_1007[] = { 23, 78, 45, 8, 32, 56, 1 };
20        int n_1007 = arr_1007.length;
21        System.out.printf("array yang belum terurut:\n");
22        for (int i_1007 = 0; i_1007 < n_1007; i_1007++)
23            System.out.print(arr_1007[i_1007] + " ");
24        System.out.println("");
25        // minMaxSelectionSort (arr, n);
26        bubbleSort_1007(arr_1007);
27        System.out.printf("array yang terurut:\n");
28        for (int i_1007 = 0; i_1007 < n_1007; i_1007++)
29            System.out.print(arr_1007[i_1007] + " ");
30        System.out.println("");
31    }
32 }
33 }
```

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma Bubble Sort. Algoritma ini bekerja dengan cara membandingkan dua elemen yang bersebelahan, kemudian menukarnya jika urutannya tidak sesuai. Proses perbandingan dan pertukaran dilakukan secara berulang hingga seluruh elemen array tersusun secara berurutan dari nilai terkecil ke terbesar.

Di dalam class BubbleSort_2511531007 terdapat method bubbleSort_1007(int[] arr_1007) yang berfungsi untuk melakukan proses pengurutan data. Variabel n_1007 digunakan untuk menyimpan jumlah elemen yang terdapat pada array. Selanjutnya terdapat perulangan for pertama yang berfungsi untuk mengatur jumlah tahap pengurutan yang dilakukan pada array.

Di dalam perulangan tersebut terdapat perulangan for kedua yang digunakan untuk membandingkan elemen-elemen yang bersebelahan. Perbandingan dilakukan menggunakan kondisi if (arr_1007[j_1007] > arr_1007[j_1007 + 1]). Jika nilai elemen sebelah kiri lebih besar daripada elemen sebelah kanan, maka kedua elemen tersebut akan ditukar posisinya agar urutannya menjadi benar.

Proses pertukaran data dilakukan dengan bantuan variabel temp_1007 sebagai variabel sementara. Nilai pada arr_1007[j_1007] terlebih dahulu disimpan ke dalam temp_1007, kemudian nilai pada arr_1007[j_1007 + 1] dipindahkan ke posisi arr_1007[j_1007]. Setelah itu nilai yang tersimpan pada temp_1007 ditempatkan ke posisi arr_1007[j_1007 + 1]. Dengan cara ini, nilai yang lebih besar akan bergerak ke bagian kanan array pada setiap tahap pengurutan, seperti gelembung yang naik ke permukaan, sehingga algoritma ini disebut Bubble Sort.

Pada method main(), dibuat sebuah array arr_1007 yang berisi data {23, 78, 45, 8, 32, 56, 1}. Program terlebih dahulu menampilkan isi array sebelum dilakukan pengurutan menggunakan perulangan for. Setelah itu method bubbleSort_1007() dipanggil untuk mengurutkan data pada array. Setelah proses pengurutan selesai, program kembali menampilkan isi array sehingga pengguna dapat melihat hasil perubahan dari data yang belum terurut menjadi data yang sudah terurut secara ascending (dari kecil ke besar).

2.2.4 Class InsertionSortGUI_2511531007

```

1 package pekan7_2511531007;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 public class InsertionSortGUI_2511531007 extends JFrame {
22     private static final long serialVersionUID = 1L;
23     private int[] array_1007;
24     private JLabel[] labelArray_1007;
25     private JButton stepButton_1007, resetButton_1007, setButton_1007;
26     private JTextField inputField_1007;
27     private JPanel panelArray_1007;
28     private JTextArea stepArea_1007;
29
30     private int i_1007 = 1, j_1007;
31     private boolean sorting_1007 = false;
32     private int stepCount_1007 = 1;
33
34     /**
35      * Create the frame.
36      */

```

```

37 public InsertionSortGUI_2511531007() {
38     setTitle("Insertion Sort Langkah per langkah");
39     setSize(750, 400);
40     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41     setLocationRelativeTo(null);
42     setLayout(new BorderLayout());
43
44     // Panel input
45     JPanel inputPanel_1007 = new JPanel(new FlowLayout());
46     inputField_1007 = new JTextField(30);
47     setButton_1007 = new JButton("Set Array");
48     inputPanel_1007.add(new JLabel("Masukkan angka (pisahkan dengan koma)"));
49     inputPanel_1007.add(inputField_1007);
50     inputPanel_1007.add(setButton_1007);
51
52     // Panel array visual
53     panelArray_1007 = new JPanel();
54     panelArray_1007.setLayout(new FlowLayout());
55
56     // Panel kontrol
57     JPanel controlPanel_1007 = new JPanel();
58     stepButton_1007 = new JButton("Langkah Selanjutnya");
59     resetButton_1007 = new JButton("Reset");
60     stepButton_1007.setEnabled(false);
61     controlPanel_1007.add(stepButton_1007);
62     controlPanel_1007.add(resetButton_1007);
63
64     // Area teks untuk log langkah-langkah
65     stepArea_1007 = new JTextArea(8, 60);
66     stepArea_1007.setEditable(false);
67     stepArea_1007.setFont(new Font("Monospaced", Font.PLAIN, 14));
68     JScrollPane scrollPane_1007 = new JScrollPane(stepArea_1007);
69
70     // Tambahkan panel ke frame
71     add(inputPanel_1007, BorderLayout.NORTH);
72     add(panelArray_1007, BorderLayout.CENTER);
73     add(controlPanel_1007, BorderLayout.SOUTH);
74     add(scrollPane_1007, BorderLayout.EAST);
75
76     // Event Set Array
77     setButton_1007.addActionListener(e -> setArrayFromInput_1007());
78
79     // Event langkah selanjutnya
80     stepButton_1007.addActionListener(e -> performStep_1007());
81
82     // Event Reset
83     resetButton_1007.addActionListener(e -> reset_1007());
84
85 }
86

```

```

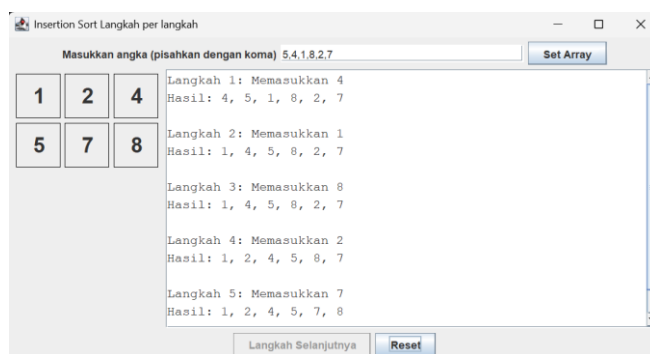
87 private void setArrayFromInput_1007() {
88     String text_1007 = inputField_1007.getText().trim();
89     if (text_1007.isEmpty())
90         return;
91     String[] parts_1007 = text_1007.split(",");
92     array_1007 = new int[parts_1007.length];
93     try {
94         for (int k_1007 = 0; k_1007 < parts_1007.length; k_1007++) {
95             array_1007[k_1007] = Integer.parseInt(parts_1007[k_1007].trim());
96         }
97     } catch (NumberFormatException e) {
98         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan " + "dengan koma", "Error",
99             JOptionPane.ERROR_MESSAGE);
100     }
101     i_1007 = 1;
102     stepCount_1007 = 1;
103     sorting_1007 = true;
104     stepButton_1007.setEnabled(true);
105     stepArea_1007.setText("");
106     panelArray_1007.removeAll();
107     labelArray_1007 = new JLabel[array_1007.length];
108     for (int k_1007 = 0; k_1007 < array_1007.length; k_1007++) {
109         labelArray_1007[k_1007] = new JLabel(String.valueOf(array_1007[k_1007]));
110         labelArray_1007[k_1007].setFont(new Font("Arial", Font.BOLD, 24));
111         labelArray_1007[k_1007].setBorder(BorderFactory.createLineBorder(Color.BLACK));
112         labelArray_1007[k_1007].setPreferredSize(new Dimension(50, 50));
113         labelArray_1007[k_1007].setHorizontalAlignment(SwingConstants.CENTER);
114         panelArray_1007.add(labelArray_1007[k_1007]);
115     }
116     panelArray_1007.revalidate();
117     panelArray_1007.repaint();
118 }
119
120 private void performStep_1007() {
121     if (i_1007 < array_1007.length && sorting_1007) {
122         int key_1007 = array_1007[i_1007];
123         j_1007 = i_1007 - 1;
124
125         StringBuilder stepLog_1007 = new StringBuilder();
126         stepLog_1007.append("Langkah ").append(stepCount_1007).append(": Memasukkan ").append(key_1007)
127             .append("\n");
128
129         while (j_1007 >= 0 && array_1007[j_1007] > key_1007) {
130             array_1007[j_1007 + 1] = array_1007[j_1007];
131             j_1007--;
132         }
133         array_1007[j_1007 + 1] = key_1007;
134
135         updateLabels_1007();
136         stepLog_1007.append("hasil: ").append(arrayToString_1007(array_1007)).append("\n\n");
137         stepArea_1007.append(stepLog_1007.toString());
138
139         i_1007++;
140         stepCount_1007++;
141
142         if (i_1007 == array_1007.length) {
143             sorting_1007 = false;
144             stepButton_1007.setEnabled(false);
145             JOptionPane.showMessageDialog(this, "Sorting selesai!");
146         }
147     }
148 }
149

```

```

151 private void updateLabels_1007() {
152     for (int k_1007 = 0; k_1007 < array_1007.length; k_1007++) {
153         labelArray_1007[k_1007].setText(String.valueOf(array_1007[k_1007]));
154     }
155 }
156
157 private void reset_1007() {
158     inputField_1007.setText("");
159     panelArray_1007.removeAll();
160     panelArray_1007.revalidate();
161     panelArray_1007.repaint();
162     stepArea_1007.setText("");
163     stepButton_1007.setEnabled(false);
164     sorting_1007 = false;
165     i_1007 = 1;
166     stepCount_1007 = 1;
167 }
168
169 private String arrayToString_1007(int[] arr_1007) {
170     StringBuilder sb_1007 = new StringBuilder();
171     for (int k_1007 = 0; k_1007 < arr_1007.length; k_1007++) {
172         sb_1007.append(arr_1007[k_1007]);
173         if (k_1007 < arr_1007.length - 1)
174             sb_1007.append(", ");
175     }
176     return sb_1007.toString();
177 }
178
179 public static void main(String[] args) {
180     SwingUtilities.invokeLater(() -> {
181         InsertionSortGUI_2511531007 gui_1007 = new InsertionSortGUI_2511531007();
182         gui_1007.setVisible(true);
183     });
184 }
185 }
186

```



Kode program di atas digunakan untuk membuat aplikasi visualisasi algoritma Insertion Sort berbasis Graphical User Interface (GUI) menggunakan library Java Swing. Program ini memungkinkan pengguna memasukkan data angka ke dalam sebuah array, kemudian melihat proses pengurutan menggunakan algoritma Insertion Sort secara bertahap melalui tombol Langkah Selanjutnya. Selain itu, program juga menampilkan setiap langkah pengurutan dalam bentuk teks sehingga proses algoritma dapat dipahami dengan lebih mudah.

Class InsertionSortGUI_2511531007 merupakan turunan dari class JFrame yang berfungsi sebagai jendela utama aplikasi. Di dalam class ini terdapat beberapa komponen GUI seperti JTextField untuk memasukkan data array, JButton untuk menjalankan perintah, JLabel untuk menampilkan elemen array secara visual, JPanel sebagai wadah komponen, serta JTextArea untuk menampilkan catatan langkah-langkah pengurutan.

Pada constructor InsertionSortGUI_2511531007(), dilakukan proses pengaturan tampilan jendela aplikasi, seperti menentukan judul, ukuran frame, posisi jendela, serta tata letak menggunakan BorderLayout. Program juga membuat panel input untuk memasukkan data array, panel visualisasi array, panel kontrol yang berisi tombol

Langkah Selanjutnya dan Reset, serta area teks untuk menampilkan proses pengurutan. Setelah itu ditambahkan event handler pada setiap tombol agar dapat menjalankan fungsi yang sesuai ketika ditekan oleh pengguna.

Method `setArrayFromInput_1007()` digunakan untuk membaca data yang dimasukkan pengguna pada `inputField_1007`. Data yang dipisahkan menggunakan tanda koma akan dipecah menjadi beberapa bagian menggunakan method `split()`, kemudian dikonversi menjadi tipe data integer dan disimpan ke dalam array `array_1007`. Jika terdapat karakter selain angka, program akan menampilkan pesan kesalahan menggunakan `JOptionPane`. Setelah data berhasil dibaca, program akan membuat label-label yang merepresentasikan setiap elemen array dan menampilkannya pada panel visualisasi.

Method `performStep_1007()` merupakan bagian utama yang menjalankan algoritma Insertion Sort secara bertahap. Pada setiap kali tombol Langkah Selanjutnya ditekan, program mengambil satu elemen yang disimpan pada variabel `key_1007`, kemudian membandingkannya dengan elemen-elemen sebelumnya menggunakan variabel `j_1007`. Jika terdapat elemen yang lebih besar dari `key_1007`, elemen tersebut akan digeser ke kanan hingga ditemukan posisi yang sesuai. Setelah proses penyisipan selesai, tampilan array diperbarui dan hasil langkah pengurutan dituliskan ke dalam `stepArea_1007`. Ketika seluruh proses pengurutan selesai, tombol langkah akan dinonaktifkan dan program menampilkan pesan bahwa proses sorting telah selesai.

Method `updateLabels_1007()` berfungsi untuk memperbarui nilai yang ditampilkan pada setiap `JLabel` sesuai dengan kondisi terbaru array setelah proses pengurutan dilakukan. Dengan demikian pengguna dapat melihat perubahan posisi data secara langsung pada tampilan GUI.

Method `reset_1007()` digunakan untuk mengembalikan aplikasi ke kondisi awal. Method ini akan menghapus isi kolom input, menghilangkan tampilan array pada panel visualisasi, mengosongkan area log langkah-langkah, menonaktifkan tombol langkah, serta mengatur ulang variabel yang digunakan selama proses pengurutan.

Method `arrayToString_1007()` berfungsi untuk mengubah isi array menjadi bentuk string yang dipisahkan dengan tanda koma. Method ini digunakan untuk menampilkan hasil array pada area log setiap kali satu langkah pengurutan selesai dilakukan.

Pada method `main()`, program dijalankan menggunakan `SwingUtilities.invokeLater()` agar seluruh komponen GUI dibuat dan dijalankan pada

Event Dispatch Thread (EDT). Selanjutnya objek InsertionSortGUI_2511531007 dibuat dan ditampilkan menggunakan method setVisible(true) sehingga pengguna dapat langsung berinteraksi dengan aplikasi visualisasi Insertion Sort tersebut.

2.3 Tugas Praktikum

2.3.1 Class Mahasiswa_2511531007

```
1 package tugasPraktikum_pekan7_2511531007;
2
3 public class Mahasiswa_2511531007 {
4     private String nama_1007;
5     private String nim_1007;
6     private String prodi_1007;
7
8     //constructor
9     public Mahasiswa_2511531007(String nama_1007, String nim_1007, String prodi_1007) {
10         this.nama_1007 = nama_1007;
11         this.nim_1007 = nim_1007;
12         this.prodi_1007 = prodi_1007;
13     }
14
15     //getter
16     public String getNama_1007() {return nama_1007;}
17     public String getNim_1007() {return nim_1007;}
18     public String getProdi_1007() {return prodi_1007;}
19
20     //setter
21     public void setNama_1007(String nama_1007) {this.nama_1007 = nama_1007;}
22     public void setNim_1007(String nim_1007) {this.nim_1007 = nim_1007;}
23     public void setProdi_1007(String prodi_1007) {this.prodi_1007 = prodi_1007;}
24
25     @Override
26     public String toString() {
27         return nama_1007;
28     }
29 }
```

Kode program di atas digunakan untuk membuat class Mahasiswa_2511531007 yang berfungsi sebagai Abstract Data Type (ADT) untuk menyimpan data mahasiswa. Class ini digunakan sebagai wadah untuk menyimpan informasi setiap mahasiswa yang nantinya akan dikelola dan diurutkan pada program sorting nama mahasiswa menggunakan GUI.

Di dalam class ini terdapat tiga atribut utama, yaitu nama_1007, nim_1007, dan prodi_1007 yang semuanya bertipe data String. Atribut nama_1007 digunakan untuk menyimpan nama mahasiswa, nim_1007 digunakan untuk menyimpan Nomor Induk Mahasiswa (NIM), sedangkan prodi_1007 digunakan untuk menyimpan program studi mahasiswa.

Kemudian terdapat constructor Mahasiswa_2511531007(String nama_1007, String nim_1007, String prodi_1007) yang berfungsi untuk membuat objek mahasiswa sekaligus menginisialisasi nilai atribut saat objek dibuat. Nilai yang diberikan melalui parameter constructor akan disimpan ke dalam atribut nama_1007, nim_1007, dan prodi_1007 sehingga setiap objek mahasiswa memiliki data yang berbeda sesuai input pengguna.

Class ini juga memiliki beberapa method getter, yaitu getNama_1007(), getNim_1007(), dan getProdi_1007(). Method getter berfungsi untuk mengambil atau

mengakses nilai atribut dari luar class tanpa harus mengakses atribut secara langsung. Dengan adanya getter, data mahasiswa dapat digunakan oleh class lain, misalnya untuk ditampilkan pada tabel atau digunakan dalam proses pengurutan data.

Selain getter, terdapat pula method setter, yaitu setName_1007(), setNim_1007(), dan setProdi_1007(). Method setter digunakan untuk mengubah atau memperbarui nilai atribut yang telah tersimpan pada objek mahasiswa. Dengan demikian data mahasiswa dapat diperbarui apabila diperlukan tanpa harus membuat objek baru.

Pada bagian akhir terdapat method toString() yang di-override dari class Object. Method ini berfungsi untuk mengembalikan nilai nama_1007 dalam bentuk String ketika objek mahasiswa ditampilkan atau dicetak. Dengan adanya method ini, objek mahasiswa dapat direpresentasikan secara sederhana menggunakan nama mahasiswa sehingga memudahkan proses penampilan data pada program.

2.3.2 Class SortingMahasiswaGUI_2511531007

```
1 package tugasPraktikum_pekan7_2511531007;
2
3 import java.awt.BorderLayout;
4 import java.awt.GridLayout;
5 import java.util.ArrayList;
6
7 import javax.swing.JButton;
8 import javax.swing.JComboBox;
9 import javax.swing.JFrame;
10 import javax.swing.JLabel;
11 import javax.swing.JOptionPane;
12 import javax.swing.JPanel;
13 import javax.swing.JScrollPane;
14 import javax.swing.JTable;
15 import javax.swing.JTextArea;
16 import javax.swing.JTextField;
17 import javax.swing.SwingUtilities;
18 import javax.swing.table.DefaultTableModel;
19
20 public class SortingMahasiswaGUI_2511531007 extends JFrame {
21
22     private static final long serialVersionUID = 1L;
23     private JTextField namaField_1007, nimField_1007, prodiField_1007; // input data mahasiswa
24     private JButton startButton_1007, deleteButton_1007, addButton_1007; // tombol kontrol
25     private JComboBox<String> cbSorting_1007; // algoritma sorting
26     private JTable tabel_1007; // menampilkan data mahasiswa
27     private DefaultTableModel model_1007;
28     private JTextArea stepArea_1007; // menampilkan langkah sorting
29     private ArrayList<Mahasiswa_2511531007> data_1007 = new ArrayList<>(); // menyimpan data mahasiswa
30 }
```

```

30
31 // Constructor GUI
32 public SortingMahasiswaGUI_2511531007() {
33     setTitle("Sorting Nama Mahasiswa");
34     setSize(900, 500);
35     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
36     setLocationRelativeTo(null);
37
38     // Panel input
39     JPanel panelInput_1007 = new JPanel(new GridLayout(4, 2));
40
41     panelInput_1007.add(new JLabel("Nama"));
42     namaField_1007 = new JTextField();
43     panelInput_1007.add(namaField_1007);
44
45     panelInput_1007.add(new JLabel("NIM"));
46     nimField_1007 = new JTextField();
47     panelInput_1007.add(nimField_1007);
48
49     panelInput_1007.add(new JLabel("Program Studi"));
50     prodiField_1007 = new JTextField();
51     panelInput_1007.add(prodiField_1007);
52
53     panelInput_1007.add(new JLabel("Algoritma"));
54
55     cbSorting_1007 = new JComboBox<>();
56     cbSorting_1007.addItem("Insertion Sort");
57     cbSorting_1007.addItem("Selection Sort");
58     cbSorting_1007.addItem("Bubble Sort");
59
60     panelInput_1007.add(cbSorting_1007);
61     add(panelInput_1007, BorderLayout.NORTH);
62
63     // Tabel mahasiswa
64     model_1007 = new DefaultTableModel(new String[] { "Nama", "NIM", "Program Studi" }, 0);
65     tabel_1007 = new JTable(model_1007);
66     add(new JScrollPane(tabel_1007), BorderLayout.CENTER);
67
68     // Area langkah sorting
69     stepArea_1007 = new JTextArea();
70     stepArea_1007.setEditable(false);
71     add(new JScrollPane(stepArea_1007), BorderLayout.EAST);
72
73     // Panel tombol
74     JPanel panelButton_1007 = new JPanel();
75     addButton_1007 = new JButton("Tambah Data");
76     deleteButton_1007 = new JButton("Hapus Data");
77     startButton_1007 = new JButton("Mulai Sorting");
78
79     panelButton_1007.add(addButton_1007);
80     panelButton_1007.add(deleteButton_1007);
81     panelButton_1007.add(startButton_1007);
82
83     add(panelButton_1007, BorderLayout.SOUTH);
84
85     // Event handling
86     addButton_1007.addActionListener(e -> addData_1007());
87
88     deleteButton_1007.addActionListener(e -> deleteData_1007());
89
90     startButton_1007.addActionListener(e -> startSorting_1007());
91
92 }

```



```

94 // Menambahkan data mahasiswa
95 private void addData_1007() {
96     String nama_1007 = namaField_1007.getText();
97     String nim_1007 = nimField_1007.getText();
98     String prodi_1007 = prodiField_1007.getText();
99     if (nama_1007.isEmpty() || nim_1007.isEmpty() || prodi_1007.isEmpty()) {
100
101         JOptionPane.showMessageDialog(this, "Lengkapi data terlebih dahulu");
102         return;
103     }
104
105     Mahasiswa_2511531007 mhs_1007 = new Mahasiswa_2511531007(nama_1007, nim_1007, prodi_1007);
106     data_1007.add(mhs_1007);
107     model_1007.addRow(new Object[] { nama_1007, nim_1007, prodi_1007 });
108
109     namaField_1007.setText("");
110     nimField_1007.setText("");
111     prodiField_1007.setText("");
112 }
113
114 // Menghapus data yang dipilih
115 private void deleteData_1007() {
116
117     int row_1007 = tabel_1007.getSelectedRow();
118
119     if (row_1007 >= 0) {
120         data_1007.remove(row_1007);
121         model_1007.removeRow(row_1007);
122     }
123 }
124
125 // Menjalankan algoritma sorting
126 private void startSorting_1007() {
127     stepArea_1007.setText("");
128     String pilihan_1007 = cboSorting_1007.getSelectedItem().toString();
129     if (pilihan_1007.equals("Insertion Sort")) {
130
131         stepArea_1007.append("=== INSERTION SORT ===\n");
132
133         insertionSort_1007();
134
135     } else if (pilihan_1007.equals("Selection Sort")) {
136
137         stepArea_1007.append("=== SELECTION SORT ===\n");
138
139         selectionSort_1007();
140
141     } else {
142
143         stepArea_1007.append("=== BUBBLE SORT ===\n");
144
145         bubbleSort_1007();
146     }
147
148     refreshTable_1007();
149 }
150

```

```

154 // ===== Insertion Sort
155 private void insertionSort_1807() {
156     for (int i_1807 = 1; i_1807 < data_1807.size(); i_1807++) {
157         // Data zaza zaza zaza zaza
158         Mahasiswa_2511531807 key_1807 = data_1807.get(i_1807);
159         int j_1807 = i_1807 - 1;
160
161         // ===== data zaza zaza zaza
162         while (j_1807 >= 0 && data_1807.get(j_1807).compareToIgnoreCase(key_1807.getName_1807()) > 0) {
163             data_1807.set(j_1807 + 1, data_1807.get(j_1807));
164             j_1807--;
165         }
166
167         // ===== data zaza zaza zaza zaza
168         data_1807.set(j_1807 + 1, key_1807);
169         stepArea_1807.append("Langkah " + i_1807 + " : " + tampilNama_1807() + "\n");
170     }
171 }
172 // ===== Selection Sort
173 private void selectionSort_1807() {
174     for (int i_1807 = 0; i_1807 < data_1807.size() - 1; i_1807++) {
175         int minIndex_1807 = i_1807;
176         // ===== data zaza zaza zaza
177         for (int j_1807 = i_1807 + 1; j_1807 < data_1807.size(); j_1807++) {
178             if (data_1807.get(j_1807).compareToIgnoreCase(data_1807.get(minIndex_1807).getName_1807()) < 0) {
179                 minIndex_1807 = j_1807;
180             }
181         }
182
183         // ===== data zaza
184         Mahasiswa_2511531807 temp_1807 = data_1807.get(i_1807);
185         data_1807.set(i_1807, data_1807.get(minIndex_1807));
186         data_1807.set(minIndex_1807, temp_1807);
187         stepArea_1807.append("Pass " + (i_1807 + 1) + " : " + tampilNama_1807() + "\n");
188     }
189 }
190 // ===== Bubble Sort
191 private void bubbleSort_1807() {
192     for (int i_1807 = 0; i_1807 < data_1807.size() - 1; i_1807++) {
193         for (int j_1807 = 0; j_1807 < data_1807.size() - i_1807 - 1; j_1807++) {
194             // ===== data zaza zaza zaza zaza
195             if (data_1807.get(j_1807).compareToIgnoreCase(data_1807.get(j_1807 + 1).getName_1807()) > 0) {
196                 Mahasiswa_2511531807 temp_1807 = data_1807.get(j_1807);
197                 data_1807.set(j_1807, data_1807.get(j_1807 + 1));
198                 data_1807.set(j_1807 + 1, temp_1807);
199             }
200             stepArea_1807.append("Pass " + (i_1807 + 1) + " : " + tampilNama_1807() + "\n");
201         }
202     }
203 }
204 // ===== data zaza zaza zaza zaza
205 private String tampilNama_1807() {
206     StringBuilder sb_1807 = new StringBuilder("");
207     for (int i_1807 = 0; i_1807 < data_1807.size(); i_1807++) {
208         sb_1807.append(data_1807.get(i_1807).getName_1807());
209         if (i_1807 < data_1807.size() - 1) {
210             sb_1807.append(", ");
211         }
212     }
213     sb_1807.append("\n");
214     return sb_1807.toString();
215 }
216 // ===== data zaza zaza zaza zaza
217 private void reverseSorting_1807() {
218     model_1807.setRowCount(0);
219     for (Mahasiswa_2511531807 m_1807 : data_1807) {
220         model_1807.addRow(new Object[] { m_1807.getName_1807(), m_1807.getNim_1807(), m_1807.getProdi_1807() });
221     }
222 }
223 // ===== program GUI
224 public static void main(String[] args) {
225     SwingUtilities.invokeLater(() -> {
226         SortingMahasiswaGUI_2511531807 gui_1807 = new SortingMahasiswaGUI_2511531807();
227         gui_1807.setVisible(true);
228     });
229 }

```

Sorting Nama Mahasiswa

Nama

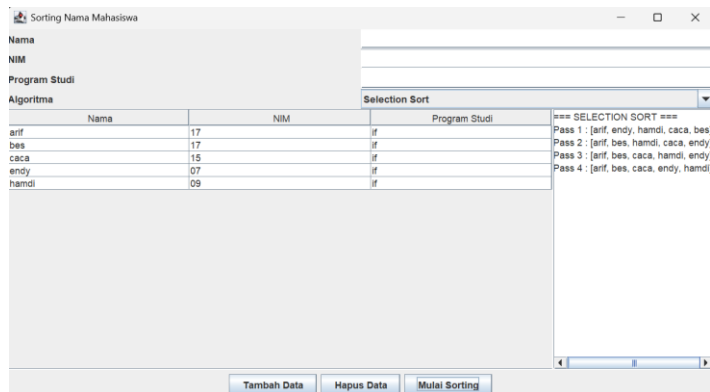
NIM

Program Studi

Algoritma

Nama	NIM	Program Studi	Algoritma
arif	17	if	=== INSERTION SORT === Langkah 1 : [arif, zaza, tifa, sis, endy] Langkah 2 : [arif, tifa, zaza, sis, endy] Langkah 3 : [arif, sis, tifa, zaza, endy] Langkah 4 : [arif, endy, sis, tifa, zaza]
endy	17	if	
sis	11	if	
tifa	07	if	
zaza	01	if	

Tambah Data Hapus Data Mulai Sorting



Kode program di atas digunakan untuk membuat aplikasi GUI pengurutan nama mahasiswa menggunakan Java Swing. Program ini memungkinkan pengguna menambahkan data mahasiswa berupa nama, NIM, dan program studi, kemudian mengurutkan data berdasarkan nama menggunakan algoritma Insertion Sort, Selection Sort, atau Bubble Sort.

Pada bagian awal program terdapat deklarasi komponen GUI seperti JTextField untuk input data mahasiswa, JButton sebagai tombol kontrol, JComboBox untuk memilih algoritma sorting, JTable untuk menampilkan data mahasiswa, dan JTextArea untuk menampilkan langkah-langkah proses sorting. Selain itu terdapat ArrayList data_1007 yang digunakan untuk menyimpan objek mahasiswa.

Constructor SortingMahasiswaGUI_2511531007() berfungsi untuk membangun tampilan GUI, mulai dari membuat form input, tabel data, area visualisasi sorting, tombol kontrol, hingga menambahkan event pada setiap tombol agar dapat menjalankan fungsi yang sesuai.

Method addData_1007() digunakan untuk menambahkan data mahasiswa ke dalam ArrayList dan JTable. Sebelum data disimpan, program akan memeriksa apakah seluruh field telah diisi. Jika ada yang kosong, program akan menampilkan pesan peringatan.

Method deleteData_1007() berfungsi untuk menghapus data mahasiswa yang dipilih pada tabel, baik dari ArrayList maupun JTable.

Method startSorting_1007() digunakan untuk menjalankan algoritma sorting sesuai pilihan pengguna pada ComboBox. Program akan memanggil method insertionSort_1007(), selectionSort_1007(), atau bubbleSort_1007() sesuai algoritma yang dipilih.

Method `insertionSort_1007()`, `selectionSort_1007()`, dan `bubbleSort_1007()` merupakan implementasi tiga algoritma sorting yang digunakan untuk mengurutkan data mahasiswa berdasarkan nama secara alfabetis. Proses perbandingan nama dilakukan menggunakan method `compareToIgnoreCase()` sehingga tidak membedakan huruf besar dan huruf kecil. Setiap langkah pengurutan ditampilkan pada `stepArea_1007` agar proses sorting dapat diamati secara bertahap.

Method `tampilNama_1007()` berfungsi untuk mengubah daftar nama mahasiswa menjadi bentuk String sehingga dapat ditampilkan pada area langkah sorting.

Method `refreshTable_1007()` digunakan untuk memperbarui isi tabel setelah proses sorting selesai sehingga urutan data pada tabel sesuai dengan hasil pengurutan.

Terakhir, method `main()` berfungsi sebagai titik awal program yang menjalankan dan menampilkan GUI menggunakan `SwingUtilities.invokeLater()`.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Insertion Sort, Selection Sort, dan Bubble Sort merupakan algoritma pengurutan data yang dapat digunakan untuk menyusun data secara terurut berdasarkan kriteria tertentu. Ketiga algoritma tersebut memiliki mekanisme yang berbeda dalam melakukan proses pengurutan, namun menghasilkan data yang telah tersusun dengan benar sesuai urutan yang diinginkan.

Pada praktikum ini berhasil dilakukan implementasi algoritma Insertion Sort, Selection Sort, dan Bubble Sort menggunakan bahasa pemrograman Java, baik dalam bentuk program sederhana maupun aplikasi berbasis GUI (Swing). Program yang dibuat mampu melakukan input data mahasiswa, menampilkan proses pengurutan secara bertahap, serta mengurutkan nama mahasiswa secara alfabetis menggunakan method `compareToIgnoreCase()`. Melalui praktikum ini, pemahaman mengenai konsep, cara kerja, dan implementasi algoritma sorting dalam pengolahan data menjadi lebih baik.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Dengan adanya latihan yang cukup, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

- [1] W. Wahyudi, *Struktur Data dengan Java*. CV Eureka Media Aksara, 2025. [Online].
Available: <https://books.google.co.id/books?id=EN2lEQAAQBAJ>.